MODULE 20 · DOMAIN 3 - DEPLOYMENT

AWS CloudFormation

Template anatomy, Ref against GetAtt, and the update behaviours that decide whether a change is safe.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 20 of 34

admin@sthithapragna.com

A template describes the **desired state**. CloudFormation works out the actions – and the risk is in what it decides to replace.

Template anatomy

SECTION	WHAT IT HOLDS
AWSTemplateFormatVersion	Almost always 2010-09-09. Optional
Description	A string describing the template. Optional
Metadata	Extra data, including cfn-init configuration
Parameters	Values supplied at deploy time
Mappings	Fixed lookup tables, keyed by Region or environment
Conditions	Boolean expressions controlling whether resources are created
Transform	Macros to apply. SAM uses this
Resources	The resources themselves. The only required section
Outputs	Values returned, and optionally exported to other stacks

ON THE EXAM

Resources is the only mandatory section. Everything else is optional, which is worth knowing because the exam asks it directly.

YAML or JSON?

- Both are fully supported and produce identical stacks.
- YAML allows **comments**, multi-line strings and the short form **!Ref** and **!GetAtt**.
- The short form cannot be nested inside another short form. Use the full **Fn::** syntax there.
- JSON is easier to generate programmatically, which is why tools emit it.
- YAML indentation errors are the most common template syntax failure. Validate before deploying.

> ON THE EXAM
>
> Exam questions use both. Recognise !GetAtt and Fn::GetAtt as the same thing, because a question may show either.

Parameters

- Typed: String, Number, List, CommaDelimitedList, and AWS-specific types.
- AWS-specific types like **AWS::EC2::KeyPair::KeyName** are validated before deployment starts.
- **AllowedValues**, **AllowedPattern**, **MinLength** and **Default** constrain what can be passed.
- **NoEcho** masks a value in the console and in events. Use it for anything sensitive.
- **SSM parameter types** fetch the current value from Parameter Store at deploy time.

> ON THE EXAM
>
> NoEcho is the security answer, and it is imperfect: the value still reaches the resource. A dynamic reference to Secrets Manager is better.

Pseudo parameters

- **AWS::Region** and **AWS::AccountId**, so a template works anywhere without editing.
- **AWS::StackName** and **AWS::StackId**, useful for naming and tagging.
- **AWS::Partition**, which is aws, aws-cn or aws-us-gov. Needed to build correct ARNs.
- **AWS::NoValue** removes a property entirely when used with Fn::If.
- **AWS::URLSuffix** differs in China, which is why hard-coded endpoints break there.

ON THE EXAM

AWS::NoValue is the clever one. Returning it from a condition deletes the property rather than setting it to null, which is how optional properties are done.

Ref

- For a **parameter**, Ref returns the value that was passed in.
- For a **resource**, Ref returns that resource's **physical ID**, whatever that means for its type.
- What Ref returns varies by resource type, and that is where mistakes happen.
- An S3 bucket returns its **name**. An EC2 instance returns its **instance ID**.
- An IAM role returns its **role name**, not its ARN. A common source of failures.

> ON THE EXAM
>
> That last line matters constantly, because most places you use a role need the ARN. Ref gives you the name, and GetAtt gives you the ARN.

Fn::GetAtt

- Returns a **named attribute** of a resource, rather than its physical ID.
- Syntax takes two arguments: the logical resource name and the attribute name.
- **!GetAtt MyRole.Arn**, **!GetAtt MyBucket.DomainName**, **!GetAtt MyDB.Endpoint.Address**.
- Nested attributes use a dot, as in the database endpoint example above.
- Available attributes are per resource type. Only the documented ones exist.

> ON THE EXAM
>
> The database endpoint is the classic case. Ref on an RDS instance gives an identifier; GetAtt Endpoint.Address gives the hostname you actually connect to.

Ref or GetAtt?

REF

The physical ID

- One argument: the logical name
- Parameter value, or resource physical ID
- Bucket name, instance ID, role name
- Queue URL for an SQS queue
- Topic ARN for an SNS topic

FN::GETATT

A named attribute

- Two arguments: resource and attribute
- Anything the resource type publishes
- ARNs, endpoints, DNS names
- Queue ARN for an SQS queue
- Nested attributes use a dot

THE TIEBREAKER

Note SNS and SQS disagree: Ref gives a topic ARN but a queue URL. That inconsistency is exactly the kind of detail the exam tests.

Fn::Sub and Fn::Join

- **Fn::Sub** substitutes variables into a string using **${...}** placeholders.
- **${MyBucket}** is a Ref. **${MyRole.Arn}** is a GetAtt, inline.
- **${!Literal}** escapes, producing a literal dollar-brace in the output.
- **Fn::Join** concatenates a list with a delimiter. Older, and far harder to read.
- Sub can do almost everything Join does, more legibly. Prefer it.

> ON THE EXAM
>
> Sub handles Ref and GetAtt inline, which is why modern templates rarely use Join. Recognise Join, because older templates and exam questions still use it.

The other intrinsic functions

- **Fn::FindInMap** looks a value up in Mappings. Three arguments: map, top key, second key.
- **Fn::ImportValue** reads a value exported by another stack.
- **Fn::If**, **Fn::Equals**, **Fn::Not**, **Fn::And**, **Fn::Or** build and use conditions.
- **Fn::Select** and **Fn::Split** pick from and produce lists. **Fn::GetAZs** lists a Region's zones.
- **Fn::Base64** encodes user data. **Fn::Cidr** carves a CIDR block into subnets.

> ON THE EXAM
>
> Select combined with GetAZs is the standard way to place subnets across availability zones without hard-coding zone names.

Mappings and Conditions

- **Mappings** are static two-level lookup tables, typically keyed by Region or environment.
- They cannot use intrinsic functions. The values must be literal.
- **Conditions** are named boolean expressions declared in their own section.
- A resource with a **Condition** attribute is created only when that condition is true.
- So one template can create a read replica in production and omit it in dev.

> ON THE EXAM
>
> Conditions plus AWS::NoValue is the combination that makes one template serve every environment without duplicating it.

Outputs and exports

- An **Output** returns a value, visible on the stack and readable by the CLI.
- Adding **Export** publishes it under a name unique across the Region.
- Another stack reads it with **Fn::ImportValue**.
- **An exported value cannot be changed or deleted while another stack imports it.**
- That creates hard coupling, and it is the main argument against cross-stack references.

> ON THE EXAM
>
> The locking is the trade-off. Exports give clean separation and make the producing stack difficult to change, which surprises teams later.

Nested stacks or cross-stack references?

NESTED STACKS

One deployment unit

- Child stacks as resources in a parent
- Deployed and deleted together
- Good for reusable components
- Values passed as parameters
- Parent must be updated to change a child

CROSS-STACK REFERENCES

Independent lifecycles

- Separate stacks, separately deployed
- Linked by Export and ImportValue
- Good for shared infrastructure
- Different teams can own each stack
- Exports lock the producing stack

THE TIEBREAKER

Nested when components share a lifecycle and are deployed as one thing. Cross-stack when a networking team owns the VPC and application teams own their own stacks.

Dependencies

- CloudFormation infers order from **Ref** and **GetAtt**. Referencing something creates a dependency.
- Resources with no dependency between them are created **in parallel**.
- **DependsOn** declares an order that cannot be inferred.
- The classic case is an EC2 instance that needs an internet gateway attachment to exist first.
- Overusing DependsOn serialises the deployment and makes it slower for no benefit.

> ON THE EXAM
>
> Most dependency problems are already solved by a Ref. Reach for DependsOn only when the ordering requirement is real but invisible to CloudFormation.

The stack lifecycle

- **CREATE_IN_PROGRESS** to **CREATE_COMPLETE**, or to **ROLLBACK_IN_PROGRESS** on failure.
- **ROLLBACK_COMPLETE** after a failed create means the stack **can only be deleted**, not updated.
- **UPDATE_ROLLBACK_COMPLETE** after a failed update leaves the previous version running.
- **UPDATE_ROLLBACK_FAILED** needs manual intervention: continue the rollback, skipping resources.
- **DELETE_FAILED** usually means a non-empty S3 bucket or a resource with a dependency outside the stack.

> ON THE EXAM
>
> ROLLBACK_COMPLETE trips everyone once. A stack in that state cannot be updated; you must delete it and create it again.

Rollback

- By default a failed operation **rolls back automatically** to the last known good state.
- **You still pay** for resources that existed during the failed attempt.
- Disabling rollback leaves failed resources in place, which is how you debug the cause.
- **Rollback triggers** monitor CloudWatch alarms during and after a deployment.
- If an alarm fires within the monitoring period, the whole stack update rolls back.

> ON THE EXAM
>
> Rollback triggers are the CloudFormation equivalent of AppConfig's alarm rollback: a deployment that looks successful can still be reverted by a metric.

Change sets

- A change set is a **preview** of what an update would do, without doing it.
- It lists each resource as **Add**, **Modify** or **Remove**.
- Crucially it shows **Replacement: True, False or Conditional**.
- You then execute the change set, or delete it and change the template.
- Change set creation also performs **early validation**, catching property errors before provisioning.

> READ THIS TWICE
>
> Replacement: True on a database is the warning a change set exists to give you. Applying a template directly gives you no such warning.

The three update behaviours

BEHAVIOUR	WHAT HAPPENS
Update with no interruption	Changed in place. The resource keeps running and keeps its physical ID
Update with some interruption	Briefly unavailable, but the same resource. An EC2 instance type change
Replacement	A new resource is created and the old one deleted. New physical ID

ON THE EXAM

Replacement is the dangerous one. Data on the old resource is gone, and anything referencing the old physical ID outside the stack now points at nothing.

DeletionPolicy

- **Delete** is the default for almost everything. The resource goes with the stack.
- **Retain** keeps the resource, removing it from CloudFormation's scope. It keeps costing money.
- **Snapshot** takes a final snapshot first, for types that support one.
- **Exception:** RDS DB clusters and standalone DB instances default to **Snapshot**, not Delete.
- **RetainExceptOnCreate** retains on delete but not when a failed create rolls back.

> ON THE EXAM
>
> That RDS exception is worth knowing precisely. Most resources default to Delete; databases default to Snapshot, which is AWS protecting you from yourself.

UpdateReplacePolicy

- Applies when an **update** replaces a resource, rather than when a stack is deleted.
- Same options: Delete, Retain, Snapshot.
- Without it, a replacement **deletes the old resource and its data**, quietly.
- Set both attributes together on anything holding data. They cover different events.
- Neither prevents replacement. They only decide what happens to the old resource.

> ON THE EXAM
>
> Setting DeletionPolicy alone is a common half-measure: it protects against stack deletion and does nothing about an update that replaces the resource.

Stack policies and termination protection

- A **stack policy** is a JSON document controlling which resources may be updated.
- Typically: deny Update:Replace and Update:Delete on the database, allow everything else.
- It applies only to **stack updates**. It does not protect against stack deletion.
- **Termination protection** blocks deletion of the whole stack until it is disabled.
- The two are complementary, and neither replaces DeletionPolicy.

> ON THE EXAM
>
> Three separate protections, each covering a different event: stack policy for updates, termination protection for deletion, DeletionPolicy for what survives either.

Drift detection

- Compares each resource's actual configuration against the template that created it.
- Reports **IN_SYNC**, **MODIFIED**, **DELETED** or **NOT_CHECKED** per resource.
- **It detects. It does not fix.** Correcting drift is a separate action.
- Drifted resources may be overwritten by the next stack update, without warning.
- Not every resource type supports drift detection, hence NOT_CHECKED.

> ON THE EXAM
>
> 'How do I find changes made outside CloudFormation' is the exam phrasing, and drift detection is the answer every time.

Importing existing resources

- **Resource import** brings an existing resource into a stack without recreating it.
- You supply a template describing it, plus its identifier, and CloudFormation adopts it.
- The template must match the real configuration, or the import fails.
- Also used to **resolve drift**: remove with Retain, then import with a corrected template.
- The **IaC generator** writes that template for you, which is why the two features pair.

> ON THE EXAM
>
> Import is how a hand-built resource becomes managed infrastructure. Without it the only option was to recreate the resource, which usually meant downtime.

StackSets

- Deploys a single template across **many accounts and many Regions** from one operation.
- Two permission models: **self-managed** with IAM roles, or **service-managed** with Organizations.
- Service-managed adds **automatic deployment** to accounts as they join an OU.
- **Operation preferences** control concurrency and a failure tolerance.
- **Stack instances** are the per-account, per-Region deployments of the stack set.

> ON THE EXAM
>
> The signal is 'across all our accounts' or 'in every Region'. A single stack cannot span either, and doing it by hand does not scale.

Custom resources

- A resource backed by a **Lambda function** or an SNS topic that you implement.
- Used for anything CloudFormation has no resource type for, or for setup steps.
- Your function receives **Create**, **Update** and **Delete** events and must handle all three.
- **It must signal back to a pre-signed URL**, or the stack hangs until it times out.
- The classic use is emptying an S3 bucket so the stack can delete it.

> ON THE EXAM
>
> The signalling requirement causes the classic symptom: a stack stuck in CREATE_IN_PROGRESS for an hour because the function threw before responding.

Helper scripts and signals

- **cfn-init** reads AWS::CloudFormation::Init metadata and installs packages, files and services.
- **cfn-signal** reports success or failure back to CloudFormation.
- **cfn-hup** watches for metadata changes and re-runs configuration.
- A **CreationPolicy** makes CloudFormation wait for a signal before marking a resource complete.
- Without a CreationPolicy, an instance is complete when it boots, not when your software is ready.

> ON THE EXAM
>
> That last line is the point. CloudFormation cannot see inside your instance, so without cfn-signal it will happily report success on a half-configured server.

Currency note: the modern tooling

- **IaC generator**, February 2024, produces a template from existing resources. Over 600 types.
- Targeted resource scans followed in March 2025, so you scan a subset rather than the account.
- **Stack refactoring**, February 2025, moves resources between stacks without recreating them.
- **Infrastructure Composer**, renamed from Application Composer in October 2024, builds templates visually.
- **Git sync** deploys a stack directly from a repository via AWS CodeConnections.

> ON THE EXAM
>
> IaC generator is the one to remember: 'we have resources created by hand and want them under CloudFormation' used to mean writing the template yourself.

Transforms

- **AWS::Serverless-2016-10-31** is the SAM transform. Module 21 covers it properly.
- **AWS::Include** pulls in a template snippet from S3.
- **AWS::LanguageExtensions** adds functions the core language lacks.
- It provides **Fn::ForEach** for loops, plus Fn::Length and Fn::ToJsonString.
- Custom **macros** are Lambda functions that rewrite a template before deployment.

> ON THE EXAM
>
> Fn::ForEach removes the classic complaint that CloudFormation has no loops. It needs the LanguageExtensions transform declared explicitly.

Diagnosing a failed stack

SYMPTOM	USUAL CAUSE
Stuck in CREATE_IN_PROGRESS for an hour	A custom resource that never signalled back
ROLLBACK_COMPLETE and cannot update	A failed create. Delete the stack and start again
DELETE_FAILED on a bucket	The bucket is not empty. CloudFormation will not empty it
Export cannot be updated	Another stack is importing it
Resource replaced unexpectedly	A property change that forces replacement. A change set would have shown it
No export named X	The producing stack has not deployed, or the name differs

ON THE EXAM

Read stack events from the bottom. The first failure is the real cause; everything above it is consequence.

Two of these turn on the same question: **will this change replace the resource**?

A team updated a production stack to change an RDS instance's database name. CloudFormation replaced the instance, creating a new empty database and deleting the original. Several hours of transactions were lost.

The team wants to be certain this cannot happen again, and wants to see the impact of a change before applying it.

What should the team adopt?

A Enable termination protection on the stack

B Enable drift detection and run it before each deployment

C Create a change set before every update, and set DeletionPolicy and UpdateReplacePolicy on the database

D Disable automatic rollback so failed updates leave resources in place

◆ PAUSE AND DECIDE

What should the team adopt?

A **Termination protection**

Protects against deleting the whole stack. This was an update, and the stack was never deleted.

B **Drift detection**

Finds changes made outside CloudFormation. The change here came through CloudFormation, so there was no drift to find.

C **Change set plus both policies**

The change set shows Replacement: True in advance, and UpdateReplacePolicy: Snapshot means the old database survives even if it happens.

D **Disable rollback**

Affects what happens after a failure. This update succeeded exactly as CloudFormation intended, which is the problem.

THE SIGNAL

Note the pairing: the change set gives warning, the policy gives protection. UpdateReplacePolicy is the one that matters, because this was a replacement, not a deletion.

> A template creates an IAM role and a Lambda function. The function's Role property is set with !Ref ExecutionRole. Stack creation fails with an error stating the role ARN is invalid.
>
> The role itself is created successfully. Only the function fails.

What is wrong?

A Ref on an IAM role returns the role name. The Role property needs !GetAtt ExecutionRole.Arn

B The role needs a DependsOn attribute so it is created first

C The role's trust policy is missing the Lambda service principal

D IAM roles cannot be referenced from a Lambda resource in the same template

◆ PAUSE AND DECIDE

What is wrong?

A Ref returns the name, use GetAtt Arn

The Role property requires an ARN. Ref on an IAM role returns the role name, so CloudFormation passes a bare name where an ARN is expected.

B Missing DependsOn

The Ref already creates an implicit dependency, and the role was created successfully. Ordering is not the issue.

C Missing trust policy principal

That would fail at invocation or produce a different error. The error explicitly says the ARN is invalid, which is a format problem.

D Cannot reference in the same template

This is the normal pattern and works fine. Templates reference roles from functions constantly.

THE SIGNAL

The error message names the problem: it says ARN. Ref returned a name, and the difference between the two is the single most tested CloudFormation detail.

> A company must deploy the same security baseline stack, containing IAM roles, a CloudTrail trail and Config rules, into 60 AWS accounts across 4 Regions. New accounts are created most weeks and must receive the baseline automatically.
>
> All accounts belong to one AWS Organization. Deploying by hand is no longer viable.

How should this be deployed?

A. A nested stack in the management account, with one child stack per target account

B. A service-managed StackSet targeting the organizational unit, with automatic deployment enabled

C. A pipeline that assumes a role in each account and runs create-stack in a loop

D. One stack in the management account, exporting values the other accounts import

◆ PAUSE AND DECIDE

How should this be deployed?

A Nested stacks

Nested stacks are created in the same account and Region as the parent. They cannot span accounts at all.

B Service-managed StackSet

Purpose-built: one template across accounts and Regions, and automatic deployment adds new accounts to the OU without any action.

C Pipeline looping create-stack

Rebuilds StackSets by hand, with 240 stacks to track, no failure tolerance and its own code to maintain.

D Exports and imports

Exports are Regional and same-account. Fn::ImportValue cannot read an export from another account.

THE SIGNAL

'New accounts must receive it automatically' is the clause that picks service-managed over self-managed. Only the Organizations model does that.

> A stack creates an S3 bucket populated by an application. Attempts to delete the stack fail with DELETE_FAILED on the bucket. A separate change also needs a one-off DynamoDB table seeded with reference data at creation time.
>
> The team wants both handled inside the template rather than by manual steps.

What should the template use?

A. DeletionPolicy: Retain on the bucket, and a Mappings section for the seed data

B. A stack policy denying deletion of the bucket, and cfn-init for the seed data

C. Termination protection, and a DependsOn between the table and the application

D. A Lambda-backed custom resource that empties the bucket on Delete and writes the seed data on Create

◆ PAUSE AND DECIDE

What should the template use?

A Retain plus Mappings

Retain avoids the failure by orphaning the bucket, which still exists and still costs money. Mappings hold literal values and cannot write to a table.

B Stack policy plus cfn-init

Stack policies govern updates, not deletion. cfn-init runs on EC2 instances, and there are none here.

C Termination protection plus DependsOn

Termination protection makes deletion harder rather than possible, and DependsOn only orders resources. Neither writes any data.

D Lambda-backed custom resource

The standard answer for both halves: emptying a bucket on Delete, and running one-off setup on Create. Both are things CloudFormation cannot do natively.

THE SIGNAL

Two requirements, one mechanism. 'CloudFormation cannot do this natively' is the custom resource signal, and emptying a bucket is its most cited example.

What to carry forward

- **Ref returns the physical ID, GetAtt returns an attribute.** A role's ARN needs GetAtt.
- **Change sets show Replacement** before it happens. That column is why they exist.
- **DeletionPolicy and UpdateReplacePolicy cover different events.** Set both on anything holding data.
- Drift detection finds out-of-band changes. It reports them; it does not fix them.
- StackSets for many accounts or Regions. Custom resources for what CloudFormation cannot do.

> ON THE EXAM
>
> Next: AWS SAM and the CDK - two higher-level ways to produce the templates this module has been reading.